

# P1467R3

## Extended floating-point types

Published Proposal, 2020-01-10

**This version:**

<https://wg21.link/P1467>

**Issue Tracking:**

[Inline In Spec](#)

**Authors:**

[David Olsen](#) (NVIDIA)

[Michał Dominiak](#) (NVIDIA)

**Audience:**

EWG, LEWG

**Toggle Diffs:**

☐ Hide deleted text

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Table of Contents

|          |                              |
|----------|------------------------------|
| <b>1</b> | <b>Abstract</b>              |
| <b>2</b> | <b>Revision history</b>      |
| 2.1      | R0 -> R1 (pre-Cologne)       |
| 2.2      | R1 -> R2 (pre-Belfast)       |
| 2.3      | R2 -> R3 (pre-Prague)        |
| <b>3</b> | <b>Motivation</b>            |
| <b>4</b> | <b>Proposal summary</b>      |
| <b>5</b> | <b>Core language changes</b> |

- 5.1 Things that aren't changing
- 5.2 Extended floating-point types
  - 5.2.1 Reasoning
  - 5.2.2 Wording
- 5.3 Conversion rank
  - 5.3.1 Reasoning
  - 5.3.2 Wording
- 5.4 Promotion
  - 5.4.1 Reasoning
  - 5.4.2 Wording
- 5.5 Implicit conversions
  - 5.5.1 Example
  - 5.5.2 Wording
- 5.6 Usual arithmetic conversions
  - 5.6.1 Example
  - 5.6.2 Wording
- 5.7 Narrowing conversions
  - 5.7.1 Same representation
  - 5.7.2 Constant values
  - 5.7.3 Wording
- 5.8 Overload resolution
  - 5.8.1 Reasoning
  - 5.8.2 Examples
  - 5.8.3 Wording
  - 5.8.4 Alternate proposal
- 5.9 Pointer conversions
  - 5.9.1 Reasoning
  - 5.9.2 Wording
- 5.10 Feature test macro

## **6 Library changes**

- 6.1 Possible new names
  - 6.1.1 Standard/extended floating-point traits
  - 6.1.2 Conversion rank trait
- 6.2 `<charconv>`

|       |                              |
|-------|------------------------------|
| 6.2.1 | Wording                      |
| 6.3   | <code>&lt;format&gt;</code>  |
| 6.3.1 | Wording                      |
| 6.4   | <code>&lt;cmath&gt;</code>   |
| 6.4.1 | Wording                      |
| 6.5   | <code>&lt;complex&gt;</code> |
| 6.5.1 | Wording                      |
| 6.6   | <code>&lt;atomic&gt;</code>  |
| 6.6.1 | Wording                      |
| 6.7   | Feature test macro           |

## References

Informative References

## Issues Index

## § 1. Abstract

This proposal allows implementations to define *extended floating-point types* in addition to the three standard floating-point types. It defines the rules for how the extended floating-point types interact with each other and with other types without changing the behavior of the existing standard floating-point types. It specifies the rules for type conversions, arithmetic conversions, promotions, and narrowing conversions. It specifies the necessary standard library support for the extended floating-point types.

The companion paper, [\[P1468\]](#), defines some standard names for commonly used floating-point formats. The end goal of these two papers is to enable the use of newer floating-point types, such as IEEE 16-bit, in standard conforming code.

## § 2. Revision history

### § 2.1. R0 -> R1 (pre-Cologne)

Applied guidance from [SG6](#) in Kona 2019:

1. Make the floating-point conversion rank not ordered between types with overlapping (but not subsetted) ranges of finite values. This makes the ranking a partial order.

2. Narrowing conversions are now based on floating-point conversion rank instead of ranges of finite values, which preserve the current narrowing conversions relations between standard floating-point types; it also interacts favorably with the rank being a partial ordering.
3. Operations that deal with floating-point types whose conversion ranks are unordered are now ill-formed.
4. The relevant parts of the guidance have been applied to the library wording section as well.

Afterwards, applied suggestions from [EWGI](#) in Kona 2019 (this modifies some of the points above):

1. Apply the suggestion to make types where one has a wider range of finite values, but a lower precision than the other, unordered in their conversion rank, and therefore make operations that mix them ill-formed. The motivating example was IEEE-754 [binary16](#) and [bfloat16](#); see [Floating-point conversion rank](#) for more details. This change also caused this paper to drop the term "range of finite values", since the modified semantics are better expressed in terms of sets of values of the types.
2. Add a change to narrowing conversions, to only allow exact conversions to happen.
3. Explicitly list parts of the language that are not changed by this paper; provide a more detailed analysis of the standard library impact.

## § 2.2. R1 -> R2 (pre-Belfast)

Changes based on feedback in Cologne from [SG6](#), [LEWGI](#), and [EWGI](#). Further changes came from further development of the paper by the authors, especially the [overload resolution](#) section.

- Revised floating-point [promotion](#) rules. Removed all promotions other than `float` to `double`. Added wording for promoting values passed to varargs functions.
- Added the section on [implicit conversions](#).
- Added the section on [overload resolution](#).
- Added the sections on [feature test macros](#).
- Added the sections about the possibility of new [library traits](#).
- Changed the wording for the `abs` function in the `<cmath>` section.
- Added constraints to the I/O streams overloads for `complex` to only support standard floating-point types.
- Added the section about possible changes to `<atomic>`.

## § 2.3. R2 -> R3 (pre-Prague)

Changes based on feedback in Belfast from [EWG](#).

- Change the [overload resolution](#) rules, removing the rule that prefers one standard conversion over another based on conversion rank. Replace it with a rule that prefers one standard conversion over another only when the two types have the same representation.
- As a result of the overload resolution change, change floating-point [promotion](#) so that any type smaller than `double` promotes to `double`.
- Allow implicit conversions between pointer types that point to floating-point types with the same representation.

## § 3. Motivation

16-bit floating-point support is becoming more widely available in both hardware (ARM CPUs and NVIDIA GPUs) and software (OpenGL, CUDA, and LLVM IR). Programmers wanting to take advantage of 16-bit floating-point support have been stymied by the lack of built-in compiler support for the type. A common workaround is to define a class type with all of the conversion operators and overloaded arithmetic operators to make it behave as much as possible like a built-in type. But that approach is cumbersome and incomplete, requiring inline assembly or other compiler-specific magic to generate efficient code.

The problem of efficiently using newer floating-point types that haven't traditionally been supported can't be solved through user-defined libraries. A possible solution of an implementation changing `float` to be a 16-bit type would be unpopular because users want support for newer floating-point types in addition to the standard types, and because users have come to expect `float` and `double` to be 32- and 64-bit types and have lots of existing code written with that assumption.

This problem is worth solving, and there is no viable solution under the current standard. So changing the core language in an extensible and backward-compatible way is appropriate. Providing a standard way for implementations to support 16-bit floating-point types will result in better code, more portable code, and wider use of those types.

This paper changes the language so that implementations can support 16-bit and other non-standard floating-point types. [\[P1468\]](#) gives well-known names to 16-bit and other commonly used floating-point types.

The motivation for the current approach of extended floating-point types comes from discussion of the previous paper [\[P0192\]](#). That proposal's single new standard type of `short float` was considered

insufficient, preventing the use of both IEEE-754 16-bit and `bfloat16` in the same application. When that proposal was rejected, the current, more expansive, proposal was developed. It is not feasible to predict which floating-point types, or even how many different types, will be used in the future, so this proposal allows for as many types as the implementation sees fit.

The language rules in this paper and the type aliases in [\[P1468\]](#) are designed to work together to simplify the safe adoption of the new floating-point types into existing applications. Programmers should be able to start using the 16-bit types in one part of the application without having to change other parts. When `float` and `double` are IEEE-conformant types, it should be possible to mix the standard types with their fixed-layout aliases without problems. This proposal would be a failure if code using the IEEE 64-bit type alias had to be kept mostly separate from code using `double`.

## § 4. Proposal summary

In a nutshell:

1. Introduce *extended floating-point types*.
2. Define *floating-point conversion rank*, which governs how floating-point types interact with each other.
3. Adjust the rules for promotion, standard conversions, usual arithmetic conversions, narrowing conversions, and overload resolution to make use of conversion rank.
4. Add function overloads and template specializations for extended floating-point types to `<cmath>`, `<charconv>`, `<format>`, `<complex>` and `<atomic>`.

## § 5. Core language changes

### § 5.1. Things that aren't changing

It is currently implementation-defined whether or not the floating-point types support infinity and NaN. That is not changing. That feature will still be implementation-defined, even for extended floating-point types.

The radix of the exponent of each floating-point type is currently implementation-defined. That is not changing. This paper will make it easier for the radix of extended floating-point types to be different from the radix of the standard types.

## § 5.2. Extended floating-point types

In addition to the three standard floating-point types, `float`, `double`, and `long double`, implementations may define any number of *extended floating-point types*, similar to how implementations may define extended integer types.

### § 5.2.1. Reasoning

The set of floating-point types that have hardware support is not possible to accurately predict years into the future. The standard needs to provide an extensible solution that can adapt to changing hardware without having to modify the standard.

### § 5.2.2. Wording

Modify 6.7.1 "Fundamental types" [**basic.fundamental**] paragraph 12:

There are three *standard floating-point types*: `float`, `double`, and `long double`. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. *There may also be implementation-defined extended floating-point types.* *The standard and extended floating-point types are collectively called floating-point types.* The value representation of floating-point types is implementation-defined. [...]

## § 5.3. Conversion rank

Define *floating-point conversion rank* to mimic in some ways the existing integer conversion rank. Floating-point conversion rank is defined in terms of the sets of values that the types can represent. If the set of values of type `T` is a strict superset of the set of values of type `U`, then `T` has a higher conversion rank than `U`. If two types have the exact same sets of values, they still have different conversion ranks; see the wording below for the exact rules. If the sets of values of two types are neither a subset nor a superset of each other, then the conversion ranks of the two types are unordered. Floating-point conversion rank forms a partial order, not a total order; this is the biggest difference from integer conversion rank.

### § 5.3.1. Reasoning

Earlier versions of this proposal used the range of finite values to define conversion rank, and had the conversion rank be a total ordering. Discussions in [SG6](#) in Kona pointed out that that definition resulted in undesirable interactions between IEEE `binary16` with 5-bit exponent and 10-bit mantissa, and `bfloat16` with 8-bit exponent and 7-bit mantissa. `bfloat16` has a much larger finite range, so it would have a higher conversion rank under the old rules. Mixing `binary16` and `bfloat16` in an arithmetic operation would result in the `binary16` value being converted to `bfloat16` despite the loss of three bits of precision. This implicit loss of precision was worrisome, so the definition of conversion rank was changed so that the usual arithmetic conversions between two floating-point values always preserves the value exactly.

For the purposes of conversion rank, infinity and NaN are treated just like any other values. If type `T` supports infinity and type `U` does not, then `U` can never have a higher conversion rank than `T`, even if `U` has a bigger range and a longer mantissa.

### § 5.3.2. Wording

Change the title of section 6.7.4 [**conv.rank**] from "~~Integer conversion rank~~" to "Conversion ranks", but leave the stable name unchanged. Insert a new paragraph at the end of the subclause:



Every floating-point type has a *floating-point conversion rank* defined as follows:

- The rank of a floating-point type T is greater than the rank of any floating-point type whose set of values is a proper subset of the set of values of T.
- The rank of `long double` is greater than the rank of `double`, which is greater than the rank of `float`.
- The rank of any standard floating-point type is greater than the rank of any extended floating-point type with the same set of values.
- The rank of any extended floating-point type relative to another extended floating-point type with the same set of values is implementation-defined, but still subject to the other rules for determining the floating-point conversion rank.
- For all floating-point types T1, T2, and T3, if T1 has greater rank than T2 and T2 has greater rank than T3, then T1 has greater rank than T3.

[ Note: The conversion ranks of extended floating-point types T1 and T2 will be unordered if the set of values of T1 is neither a subset nor a superset of the set of values of T2. This can happen when one type has both a larger range and a lower precision than the other. -- end note ]. [ Note: The floating-point conversion rank is used in the definition of the usual arithmetic conversions ([`expr.arith.conv`]). -- end note ]

## § 5.4. Promotion

All floating-point types with a conversion rank that is less than the rank of `double` promote to `double`. (This automatically covers arguments passed to the elipsis part of a varargs function.)

### § 5.4.1. Reasoning

This most closely matches the integer promotion rules, though the floating-point rules are simpler due to the lack of signed/unsigned and enumeration types.

### § 5.4.2. Wording

Change section 7.3.7 "Floating-point promotion" [`conv.fpprom`] as follows:

- 1 A prvalue of ~~type float~~ a floating-point type whose floating-point conversion rank (`conv.rank`) is less than the rank of `double` can be converted to a prvalue of type `double`. The value is unchanged.
- 2 This conversion is called *floating-point promotion*.

## § 5.5. Implicit conversions

The standard currently allows implicit conversions between any arithmetic types, even if the conversion could result in a loss of information. This can't be changed for any existing arithmetic types, but it is possible to choose a different behavior for the new extended floating-point types. A reasonable rule would be to allow implicit conversions between floating-point types only when converting to a type with a higher conversion rank (or when converting between two standard floating-point types, for backward compatibility).

If implicit conversions are always allowed, that most closely matches existing behavior and will likely lead to fewer surprises. If potentially lossy conversions are not implicit, that will lead to safer code. Since explicit conversions between all floating-point types would still be allowed, potentially lossy conversions would be more verbose rather than forbidden.

This issue was discussed in EWG in Belfast. There were strong opinions on both sides of the issue. The poll that was taken did not show consensus in either direction. The authors of the paper are undecided on which choice is best. So this is still an open issue.

This issue is mostly independent from the rest of the paper, so it could be decided either way without invalidating the rest of the proposal. The only area that would be affected would be overload resolution, since some standard conversions would no longer be standard conversions if implicit versions were restricted.

**ISSUE 1** Should implicit conversions be allowed from larger floating-point types to smaller floating-point types?

### § 5.5.1. Example

Assuming that extended floating-point conversions are restricted as proposed:

```

double f64 = 1.0;
float f32 = 2.0;
__fp16 f16 = 3.0;
fp64 = fp32; // okay
fp32 = fp64; // okay, standard types for backward compatibility
fp64 = fp16; // okay
fp16 = fp64; // error, implicit conversion not allowed
fp16 = static_cast<__fp16>(fp64); // okay, explicit cast

```

### § 5.5.2. Wording

If it is decided that implicit conversions are always allowed, then no wording changes are necessary.  
7.3.9 [conv.double] already does the right thing.

If it is decided that implicit conversions should be restricted, then the following wording changes are necessary:

Modify section 7.3.9 "Floating-point conversions" [conv.double] as follows:

A prvalue of floating-point type can be converted to a prvalue of another floating-point type with a higher conversion rank or with the same set of values, or a prvalue of standard floating-point type can be converted to a prvalue of another standard floating-point type. If the source value can be exactly represented in the destination type, the result of the conversion is that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined.

The conversions allowed as floating-point promotions are excluded from the set of floating-point conversions.

In section 7.6.1.8 "Static cast" [expr.static.cast], add a new paragraph after paragraph 10 ("A value of integral or enumeration type can [...]"):

A value of floating-point type can be explicitly converted to any other floating-point type. If the source value can be exactly represented in the destination type, the result of the conversion is that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined.

Note: A `static_cast` from a higher floating-point conversion rank to a lower conversion rank is already covered by [expr.static.cast] p7, which talks about inverses of standard conversions. The new paragraph is necessary to allow explicit conversions between types with unordered conversion ranks. The wording about what to do with the value is stolen from the floating-point conversions section [conv.double].

## § 5.6. Usual arithmetic conversions

The proposed usual arithmetic conversions for floating-point types are based on the floating-point conversion rank, similar to integer arithmetic conversions. But because floating-point conversions are a partial ordering, there may be some expressions where neither operand will be converted to the other's type. It is proposed that these situations are ill-formed.

### § 5.6.1. Example

In this implementation, let `float` be IEEE `binary32`, `__fp16` be IEEE `binary16`, and `__bfloat` be 16-bit bfloat.

```
float f32 = 1.0;
__fp16 f16 = 2.0;
__bfloat b16 = 3.0;
f32 + f16; // okay, f16 converted to float, result type is float
f32 + b16; // okay, b16 converted to float, result type is float
f16 + b16; // error, neither type can convert to the other via arithmetic
```

### § 5.6.2. Wording

Modify section 7.4 Usual arithmetic conversions [expr.arith.conv] as follows:

- 1 Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:
  - (1.1) - If either operand is of scoped enumeration type ([dcl.enum]), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
  - (1.2) - ~~If either operand is of type long double, the other shall be converted to long double.~~
  - (1.3) - ~~Otherwise, if either operand is double, the other shall be converted to double.~~
  - (1.4) - ~~Otherwise, if either operand is float, the other shall be converted to float.~~
  - (1.5) - Otherwise, if either operand has a floating-point type, the following rules shall be applied:
    - (1.1) - If both operands have the same type, no further conversion is needed.
    - (1.2) - Otherwise, if one of the operands has a type that is not a floating-point type, that operand shall be converted to the type of the operand with the floating-point type.
    - (1.3) - Otherwise, if the floating-point conversion ranks ([conv.rank]) of the types of the operands are ordered, then the operand with the type of the lower floating-point conversion rank shall be converted to the type of the other operand.
    - (1.4) - Otherwise, the expression is ill-formed.
  - (1.6) - Otherwise, the integral promotions ([conv.prom]) shall be performed on both operands.(59) Then the following rules shall be applied to the promoted operands:
    - (1.1) - If both operands have the same type, no further conversion is needed.
    - (1.2) - Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank shall be converted to the type of the operand with greater rank.

- (1.3) - Otherwise, if the operand that has unsigned integer type has rank greater than or equal to the rank of the type of the other operand, the operand with signed integer type shall be converted to the type of the operand with unsigned integer type.
  - (1.4) - Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, the operand with unsigned integer type shall be converted to the type of the operand with signed integer type.
  - (1.5) - Otherwise, both operands shall be converted to the unsigned integer type corresponding to the type of the operand with signed integer type.
- 2 If one operand is of enumeration type and the other operand is of a different enumeration type or a floating-point type, this behavior is deprecated (D.1).

## § 5.7. Narrowing conversions

A narrowing conversion is a conversion from a type with a higher floating-point conversion rank to a type with a lower conversion rank, or a conversion between two types with unordered conversion rank.

### § 5.7.1. Same representation

When two different floating-point types have the same representation, one of the types has a higher conversion rank than the other. Which means that a conversion between the two types will be a narrowing conversion in one of the directions even though the value will be preserved. For example, on some implementations, `double` and `long double` have the same representation, but `long double` always has a higher conversion rank than `double`, so a conversion from `long double` to `double` is considered a narrowing conversion.

An earlier version of this paper defined narrowing conversions in terms of sets of representable values, not in terms of conversion rank. With that definition, conversions between types with the same representation would never be a narrowing conversion. [SG6](#) in Kona preferred using conversion rank over sets of values, so the proposal was changed to the current definition. One argument against the old definition was that it changed the behavior for standard floating-point types, as in the example of `double` and `long double` above.

It would be possible to have different rules for standard floating-point types and extended floating-point types, but the authors feel it is best to maintain consistency between standard and extended types, and to not change the behavior of standard types.

### § 5.7.2. Constant values

This proposal preserves the existing wording in [dcl.init.list] p7.2, "except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly)." A reasonable argument could be made that this constant value exception should not apply to extended floating-point types. But the authors are not in favor of that change. It would introduce an inconsistency between standard and extended types. It would cause `__fp16 x{2.1};` to be a narrowing conversion because 2.1 cannot be represented exactly in binary floating-point representations (assuming that `__fp16` is the name of an extended floating-point type with a conversion rank lower than `double`).

### § 5.7.3. Wording

Modify the definition of narrowing conversions in 9.3.4 "List-initialization" [dcl.init.list] paragraph 7 item 2:

- ~~from long double to double or float, or from double to float~~ from a floating-point type T to another floating-point type whose floating-point conversion rank is not greater than that of T, except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly), or

## § 5.8. Overload resolution

When comparing standard conversion sequences that involve floating-point conversions, prefer conversions between types that have the same representation.

### § 5.8.1. Reasoning

The extended floating-point types should behave as much as possible as other arithmetic types, with one exception: overload resolution should prefer cases where the argument and the parameter have the same representation, even when they are different types.

Overload resolution of floating-point types behaves mostly like the `ints`, with anything smaller than `double` promoted to `double`. The difference comes when choosing among standard conversions. Unlike the `ints`, where there is no preference among standard conversions, a conversion between two floating-point types with the same representation is preferred over a conversion between types with different representations.

These rules ease the adoption of the fixed-layout type aliases defined in [\[P1468\]](#) as described at the end of the [motivation](#).

### § 5.8.2. Examples

These examples assume that `float` and `double` are IEEE 32- and 64-bit types, that `long double` is X87 80-bit, and that `__fp16`, `__fp32`, `__fp64`, `__fp128`, and `__bfloat` are all extended floating-point types (distinct from `float` and `double`) representing IEEE 16-bit, 32-bit, 64-bit, and 128-bit types and `bfloat16`.

Given that function `f` is overloaded on all the standard floating-point types:

```
void f(float);  
void f(double);  
void f(long double);
```

Calling `f` with an argument of type `float`, `double`, or `long double` will of course choose the overload with the exact match. Calling `f` with an argument of type `__fp16`, `__bfloat`, `__fp32`, or `__fp64` will choose `f(double)` because all of those types promote to `double`. Calling `f` with an argument of type `__f128` will be ambiguous because there is no exact match or promotion and none of the standard conversions is preferred over the others.

Given that function `g` is overloaded on the 32-bit and 64-bit extended types:



```
void g(__fp32);  
void g(__fp64);
```

Calling `g` with `__fp32` or `__fp64` will choose the overload with the exact match. Calling `g` with `float` will call `g(__fp32)` because `float` and `__fp32` have the same representation. For similar reasons, calling `g` with `double` will call `g(__fp64)`. Calling `g` with an argument of any other type will be ambiguous because the standard conversion to neither parameter is preferred over the other.

### § 5.8.3. Wording

In 12.3.3.2 "Ranking implicit conversion sequences" [over.ics.rank] paragraph 4, add a new bullet between (4.2) and (4.3):

- (4.2) A conversion that promotes an enumeration whose underlying type is fixed to its underlying type is better than one that promotes to the promoted underlying type, if the two are different.
- (4.3) A conversion from floating-point type FP1 to floating-point type FP2 is better than a conversion from FP1 to floating-point type FP3 if
  - (4.3.1) FP1 and FP2 have the same set of values, and
  - (4.3.2) FP1 or FP2 is an extended floating-point type, and
  - (4.3.3) FP3 has a different set of values from FP1 or the floating-point conversion rank ([conv.rank]) of FP3 is not less than the rank of FP2.
- ~~(4.3)~~ (4.4) If class B is derived directly or indirectly from class A, conversion of B\* to A\* is better than conversion of B\* to void\*, and conversion of A\* to void\* is better than conversion of B\* to void\*.

Note: The important parts of the proposed wording are (4.3.1) and the first half of (4.3.3). (4.3.2) and the second half of (4.3.3) exist to give reasonable behavior when at least two of the standard floating-point types have the same representation (which is true for `double` and `long double` on many implementations).

#### § 5.8.4. Alternate proposal

This paper contained a different set of rules for overload resolution in R2. That proposal had some opposition when presented in Belfast (though no poll was taken about it), so the overload rules were revised to what is listed immediately above. But the authors feel that the older rules have some advantages. So the older rules are listed here in case anyone can think of a way to combine the two into a new proposal that has the advantages of both.

When comparing conversion sequences that involve floating-point conversions, prefer conversions that are value-preserving, and prefer conversions to lower conversion ranks over conversions to higher conversion ranks.

This has the advantage that, when code overloads a function on some of the floating-point types, then calls to that function will be well-formed as long as the argument is of a floating-point type that can be safely converted to at least one of the possible parameter types.

For example, let `float` be IEEE 32-bit, `double` be IEEE 64-bit, and `long double` be X87 80-bit. And let `__fp16`, `__fp32`, and `__fp64` be extended floating-point types that represent IEEE 16-bit, 32-bit, and 64-bit respectively. Then given a function overloaded on the 32-bit and 64-bit extended floating-point types:

```
void f(__fp32);  
void f(__fp64);
```

The following functions calls should be well-formed:

```
f((__fp16)1.0); // calls f(__fp32)  
f((__fp32)2.0); // calls f(__fp32)  
f((__fp64)3.0); // calls f(__fp64)  
f((float)4.0);  // calls f(__fp32)  
f((double)5.0); // calls f(__fp64)
```

But the function call `f((long double)6.0);` would be an ambiguous function call.

The disadvantage of this proposal (and the reason for the opposition to it in Belfast) is that adding a new overload of an existing function can change the function that is called without the new overload being an exact match for the argument type.

In 12.3.3.2 "Ranking implicit conversion sequences" [**over.ics.rank**] paragraph 4, add a new bullet between (4.2) and (4.3):

- (4.2) A conversion that promotes an enumeration whose underlying type is fixed to its underlying type is better than one that promotes to the promoted underlying type, if the two are different.
- (4.3) A conversion from floating-point type F1 to floating-point type F2 is better than a conversion from F1 to floating-point type F3 if the set of values of F1 is a subset of the set of values of F2 and F3 has greater floating-point conversion rank (`[conv.rank]`) than F2.
- ~~(4.3)~~ (4.4) If class B is derived directly or indirectly from class A, conversion of B\* to A\* is better than conversion of B\* to void\*, and conversion of A\* to void\* is better than conversion of B\* to void\*.

## § 5.9. Pointer conversions

Pointers to two different floating-point types can be freely and implicitly converted between each other as long as the two floating-point types have the same representation.

### § 5.9.1. Reasoning

These pointer conversions will ease the transition to the fixed-layout aliases. There is lots of existing floating-point code that uses pointer-to-`float` or pointer-to-`double` as function parameters. When compilers implement `std::float64_t` (or whatever name is chosen as the name for the IEEE 64-bit type in [P1468]), users on systems where `double` is IEEE 64-bit can change their parameters and variables from `double` to `std::float64_t` incrementally. With the pointer conversions and [overload resolution](#) rules above, `double` and `std::float64_t` will essentially behave as if they were the same type even though they are different types. Users do not have to change their code from `double` to `std::float64_t` all at once and don't have to coordinate the change with third-party library vendors. (Changing code to use `std::float64_t` instead of `double` is a good thing for many users because it more clearly communicates the author's intent.)

If the user is on a system where `double` is not IEEE 64-bit (or later ports to such a system), then `(double *)` will not implicitly convert to `(std::float64_t *)`. In that environment the switch from `double` to `std::float64_t` has to be well-coordinated and can't be done piecemeal. The compiler will help with that by reporting a compilation error when such implicit pointer conversions are attempted.

If these pointer conversions are not implicit, then a user switching code from `double` to `std::float64_t` would likely have to add `static_cast`s to the code in some places. In addition to being more work, this leaves the code more fragile and error prone, because there will be runtime failures rather than compilation errors if the code is later ported to a system where `double` and `std::float64_t` do not have the same representation.

### § 5.9.2. Wording

Add a new paragraph to the end of section 7.3.11 "Pointer conversions" [**conv.ptr**]:

A prvalue of type "pointer to cv F1", where F1 is a floating-point type, can be converted to a prvalue of type "pointer to cv F2", where F2 is a different floating-point type with the same set of values as F1. The pointer value is unchanged by this conversion.

### § 5.10. Feature test macro

**ISSUE 2** Should there be a feature test macro to indicate that the implementation supports at least one extended floating-point type?

Implementations could support extended floating-point types without supporting any of the aliases defined in [P1468]. So it might be useful to have a feature test macro that indicates support for extended floating-point types listed in 15.10 [**cpp.predefined**]. But it would likely have to be one of the conditionally-defined macros, and not listed in Table 17, since a conforming compiler might choose to not define any extended floating-point types. If the macro is defined, it would not indicate which extended floating-point types are supported, only that there exists at least one extended floating-point type in the implementation.

## § 6. Library changes

Making extended floating-point types easy to use does not require introducing any new names to the standard library. But it does require adding new overloads or new template specializations in several places. (The companion paper, [P1468], does add new names related to floating-point types to the standard library. But those names are not necessary to make extended floating-point types useful.)

To handle I/O of extended floating-point types, changes are proposed to `<charconv>` and `<format>`, but not to `<iostream>` or `<cstdio>`.

Implementations will have to change `std::numeric_limits` and `std::is_floating_point` to give correct answers for extended floating-point types. The existing wording in the standard already covers that (by referring to all *floating-point types* without listing them explicitly), so no wording changes are needed.

Most of the standard functions that operate on floating-point types need wording changes to add overloads or template specializations for the extended floating-point types. These classes and functions are in `<cmath>`, `<complex>`, and `<atomic>`.

No changes are proposed to the following parts of the standard library:

- `<cfloat>`: The header `<cfloat>` provides macros describing some of the properties of the standard floating-point types. The use of macros does not extend very well to extended floating-point types with implementation-specific names. No changes are proposed to `<cfloat>`; users should use `std::numeric_limits` instead to query the properties of extended floating-point types.
- The `printf` and `scanf` families of functions: There is no practical way to add format specifiers for implementation-specific types with implementation-specific names.
- I/O streams: To support extended floating-point types, new virtual functions would need to be added to `num_get` and `num_put`, which would be an ABI break.
- The `strtod` and `stod` families of functions: With different names for each floating-point type (which for `strtod` was inherited from C), that scheme doesn't work well for extended floating-point types.
- The `std::to_string` family of functions: They are defined in terms of `snprintf`, which will not support extended floating-point types.
- `<random>`: [rand.req] states that certain template arguments have to be `float`, `double`, or `long double`. The wording could be changed to allow any floating-point type, but `<random>` does not support extended integral types, so we are not proposing that it support extended floating-point types either.

## § 6.1. Possible new names

While no new names need to be added to the standard library for extended floating-point types to be useful, there are some new things that could be useful. The authors are undecided if these are useful enough to be worth adding, and would appreciate LEWG feedback on the matter.

### § 6.1.1. Standard/extended floating-point traits

`std::is_floating_point_v<T>` is true for both standard and extended floating-point types. Should the standard also provide `std::is_standard_floating_point` and/or `std::is_extended_floating_point`? Will users need to distinguish between standard and extended types often enough that `std::is_same_v<T, float> || std::is_same_v<T, double> || std::is_same_v<T, long double>` becomes too unwieldy?

**ISSUE 3** Should the new type traits `std::is_standard_floating_point` and/or `std::is_extended_floating_point` be introduced?

### § 6.1.2. Conversion rank trait

Should there be a type trait that reports whether or not one floating-point type has a higher conversion rank than another? This could be useful when writing function templates to figure out which conversions between different floating-point types are safe. See the [constructors](#) for `std::complex` as an example of where this trait would be useful.

**ISSUE 4** Should a new type trait be introduced that can be used to query the floating-point conversion rank relationship?

## § 6.2. `<charconv>`

Add overloads for all extended floating-point types for the functions `to_chars` and `from_chars`.

### § 6.2.1. Wording

Add a new paragraph to the beginning of 20.19.1 "Header `<charconv>` synopsis" [`charconv.syn`], before the start of the synopsis:

When a function has a parameter of type *integral*, the implementation provides overloads for all signed and unsigned integer types and `char` as the parameter type. When a function has a parameter of type *floating-point*, the implementation provides overloads for all floating-point types as the parameter type.

Change the header synopsis in [`charconv.syn`] as follows:

```

to_chars_result to_chars(char* first, char* last, see belowintegral va
to_chars_result to_chars(char* first, char* last, floatfloating-point
to_chars_result to_chars(char* first, char* last, double value);
to_chars_result to_chars(char* first, char* last, long double value);
to_chars_result to_chars(char* first, char* last, floatfloating-point
                           chars_format fmt);
to_chars_result to_chars(char* first, char* last, double value, chars_
to_chars_result to_chars(char* first, char* last, long double value, c
to_chars_result to_chars(char* first, char* last, floatfloating-point
                           chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, double value,
                           chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, long double value,
                           chars_format fmt, int precision);

// ...

from_chars_result from_chars(const char* first, const char* last,
                             see belowintegral& value, int base = 10);

from_chars_result from_chars(const char* first, const char* last, floa
                           chars_format fmt = chars_format::general)
from_chars_result from_chars(const char* first, const char* last, doub
                           chars_format fmt = chars_format::general)
from_chars_result from_chars(const char* first, const char* last, long
                           chars_format fmt = chars_format::general)

```

In 20.19.2 "Primitive numeric output conversion" [**charconv.to.chars**], leave the first three paragraphs unchanged, but modify the rest of the section as follows:

`to_chars_result to_chars(char* first, char* last, see belowintegral value`

**Requires** **Expects** : base has a value between 2 and 36 (inclusive).

**Effects**: The value of `value` is converted to a string of digits in the given base (with no redundant leading zeroes). Digits in the range 10..35 (inclusive) are represented as lowercase characters `a..z`. If `value` is less than zero, the representation starts with `' - '`.

**Throws**: Nothing.

**Remarks**: **[ Note**: The implementation ~~shall provide~~ **provides** overloads for all signed and unsigned integer types and `char` as the type of the parameter `value`. ~~- end note ]~~

`to_chars_result to_chars(char* first, char* last, floatfloating-point va  
to_chars_result to_chars(char* first, char* last, double value);  
to_chars_result to_chars(char* first, char* last, long double value);`

**Effects**: `value` is converted to a string in the style of `printf` in the "C" locale. The conversion specifier is `f` or `e`, chosen according to the requirement for a shortest representation (see above); a tie is resolved in favor of `f`.

**Throws**: Nothing.

**[ Note**: The implementation **provides** overloads for all floating-point types as the type of the parameter `value`. ~~- end note ]~~

`to_chars_result to_chars(char* first, char* last, floatfloating-point va  
to_chars_result to_chars(char* first, char* last, double value, chars_fo  
to_chars_result to_chars(char* first, char* last, long double value, cha`

**Requires** **Expects** : `fmt` has the value of one of the enumerators of `chars_format`.

**Effects**: `value` is converted to a string in the style of `printf` in the "C" locale.

**Throws**: Nothing.

**[ Note**: The implementation **provides** overloads for all floating-point types as the type of the parameter `value`. ~~- end note ]~~



```
to_chars_result to_chars(char* first, char* last, floatfloating-point va  
                           chars_format fmt, int precision);  
to_chars_result to_chars(char* first, char* last, double value,  
                           chars_format fmt, int precision);  
to_chars_result to_chars(char* first, char* last, long double value,  
                           chars_format fmt, int precision);
```

**Requires** **Expects** : `fmt` has the value of one of the enumerators of `chars_format`.

**Effects**: `value` is converted to a string in the style of `printf` in the "C" locale with the given precision.

**Throws**: Nothing.

[ Note: The implementation provides overloads for all floating-point types as the type of the parameter `value`. - end note ]

See also: ISO C 7.21.6.1

Modify 20.19.3 "Primitive numeric input conversion" [**charconv.from.chars**] as follows:

- 1 All functions named `from_chars` analyze the string `[first, last)` for a pattern, where `[first, last)` is required to be a valid range. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `errc::invalid_argument`. [ *Note*: If the pattern allows for an optional sign, but the string has no digit characters following the sign, no characters match the pattern. — *end note* ] Otherwise, the characters matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `last` if all characters match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`. Otherwise, `value` is set to the parsed value, after rounding according to `round_to_nearest`, and the member `ec` is value-initialized.

```
from_chars_result from_chars(const char* first, const char* last,  
                             see belowintegral& value, int base = 10);
```

- 2 **Requires** **Expects** : `base` has a value between 2 and 36 (inclusive).
- 3 *Effects*: The pattern is the expected form of the subject sequence in the "C" locale for the given nonzero base, as described for `strtoul`, except that no `"0x"` or `"0X"` prefix shall appear if the value of `base` is 16, and except that `'-'` is the only sign that may appear, and only if `value` has a signed type.
- 4 *Throws*: Nothing.
- 5 **Remarks**: [ *Note*: The implementation **shall provide** **provides** overloads for all signed and unsigned integer types and `char` as the referenced type of the parameter `value`. — *end note* ]

```
from_chars_result from_chars(const char* first, const char* last, float f,  
                             chars_format fmt = chars_format::general);  
from_chars_result from_chars(const char* first, const char* last, double  
                             chars_format fmt = chars_format::general);  
from_chars_result from_chars(const char* first, const char* last, long d  
                             chars_format fmt = chars_format::general);
```

- 6 **Requires** **Expects** : `fmt` has the value of one of the enumerators of `chars_format`.
- 7 *Effects*: The pattern is the expected form of the subject sequence in the "C" locale, as described for `strtod`, except that

- (7.1) - the sign '+' may only appear in the exponent part;
- (7.2) - if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;
- (7.3) - if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and
- (7.4) - if `fmt` is `chars_format::hex`, the prefix "0x" or "0X" is assumed. [ *Example: The string 0x123 is parsed to have the value 0 with remaining characters x123. - end example* ]

In any case, the resulting `value` is one of at most two floating-point values closest to the value of the string matching the pattern.

8 *Throws:* Nothing.

9 [ *Note: The implementation provides overloads for all floating-point types as the referenced type of the parameter `value`. - end note* ]

See also: ISO C 7.22.1.3, 7.22.1.4

## § 6.3. `<format>`

Change `std::format` to support extended floating-point types.

### § 6.3.1. Wording

... to be determined ...

## § 6.4. `<cmath>`

Add overloads for extended floating-point types to the functions in `<cmath>`. It is expected that this will be the most used part of the library changes.

### § 6.4.1. Wording

Modify 26.8.1 "Header `<cmath>` synopsis" [**`cmath.syn`**] paragraph 2 as follows:

For each set of overloaded functions within `<cmath>`, with the exception of `abs`, there shall be additional overloads sufficient to ensure:

- ~~1. If any argument of arithmetic type corresponding to a `double` parameter has type `long double`, then all arguments of arithmetic type (6.7.1) corresponding to `double` parameters are effectively cast to `long double`.~~
- ~~2. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has type `double` or an integer type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.~~
- ~~3. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.~~
- 1. If any argument corresponding to a `double` parameter has floating-point type, then all arguments of arithmetic type ([basic.fundamental]) corresponding to `double` parameters are effectively cast to the floating-point type with the highest floating-point conversion rank ([conv.rank]) among the types of such floating-point arguments. If two such floating-point arguments have types whose conversion rank is unordered, the program is ill-formed.
- 2. Otherwise, all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.

[ Note: `abs` is exempted from these rules in order to stay compatible with C. -- end note ]

Modify section 26.8.2 "Absolute values" [`c.math.abs`] as follows:

- 1 [ *Note*: The headers `<cstdlib>` and `<cmath>` declare the functions described in this subclause. — *end note* ]

```
int abs(int j);
long int abs(long int j);
long long int abs(long long int j);
float abs(float j);
double abs(double j);
long double abs(long double j);
```

- 2 *Effects*: The `abs` functions that take integer arguments have the semantics specified in the C standard library for the functions `abs`, `labs`, and `llabs`, ~~`fabsf`, `fabs`, and `fabsl`~~.
- 3 *Remarks*: If `abs ( )` is called with an argument of type `X` for which `is_unsigned_v<X>` is `true` and if `X` cannot be converted to `int` by integral promotion, the program is ill-formed. [ *Note*: Arguments that can be promoted to `int` are permitted for compatibility with C. — *end note* ]

`floating-point abs(floating-point x);`

- 4 *Returns*: The absolute value of `x`.
- 5 *Remarks*: The implementation provides overloads for all floating-point types as the type of parameter `x`, with the same floating-point type as the return type.

See also: ISO C 7.12.7.2, 7.22.6.1

## § 6.5. `<complex>`

Make `std::complex<T>` be well-defined when `T` is an extended floating-point type. The explicit specializations of `std::complex<T>` are removed. The only differences between the explicit specializations was the explicit-ness of the constructors that take a complex number of a different type. This behavior is incorporated into the main template through `explicit (bool)`.

### § 6.5.1. Wording

Modify 26.4 "Complex numbers" [`complex.numbers`] paragraph 2 as follows:

The effect of instantiating the template `complex` for any type ~~other than float, double, or long double~~ that is not a floating-point type is unspecified. The specializations ~~`complex<float>`, `complex<double>`, and `complex<long double>`~~ of `complex` for floating-point types are literal types ([basic.types]).

Delete the explicit specializations from 26.4.1 "Header `<complex>` synopsis" [**complex.syn**]:

```
namespace std {  
    // 26.4.2, class template complex  
    template class complex;  
  
    // 26.4.3, specializations  
    template<> class complex;  
    template<> class complex;  
    template<> class complex;  
  
    // ...
```

In 26.4.2 "Class template `complex`" [**complex**], modify the synopsis of the constructors as follows:

```
constexpr complex(const T& re = T(), const T& im = T());  
constexpr complex(const complex&) = default;  
template<class X> constexpr explicit(see below) complex(const complex<X>
```

Remove section 26.4.3 "Specializations" [**complex.special**] in its entirety.

In 26.4.4 "Member functions" [**complex.members**], add the following after paragraph 2:

```
template<class X> constexpr explicit(see below) complex(const complex<X>
```

*Ensures:* `real(.) == other.real() && imag(.) == other.imag().`

*Remarks:* The expression inside `explicit` evaluates to false if and only if the floating-point conversion rank of `T` is greater than the floating-point conversion rank of `X`.

In 26.4.6 "Non-member operations" [**complex.ops**], change the streaming operators as follows:

```
template<class T, class CharT, class traits>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&
```

Constraints: T is a standard floating-point type.

~~Requires~~ Expects : The input values ~~shall be~~ are convertible to T.

*Effects:* Extracts a complex number x of the form: u, (u), or (u,v), where u is the real part and v is the imaginary part (29.7.4.2).

If bad input is encountered, calls `is.setstate(ios_base::failbit)` (which may throw `ios::failure` (29.5.5.4)).

*Returns:* is.

*Remarks:* This extraction is performed as a series of simpler extractions. Therefore, the skipping of whitespace is specified to be the same for each of the simpler extractions.

```
template<class T, class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&
```

Constraints: T is a standard floating-point type.

*Effects:* Inserts the complex number x ...

Modify 26.4.9 "Additional overloads" [**cmplx.over**] paragraphs 2 and 3 as follows:

The additional overloads shall be sufficient to ensure:

- ~~If the argument has type `long double`, then it is effectively cast to `complex<long double>`.~~
- ~~Otherwise, if the argument has type `double` or an integer type, then it is effectively cast to `complex<double>`.~~
- ~~Otherwise, if the argument has type `float`, then it is effectively cast to `complex<float>`.~~
- If the argument has a floating-point type `T`, then it is effectively cast to `complex<T>`.
- Otherwise, if the argument has integer type, then it is effectively cast to `complex<double>`.

Function template `pow` shall have additional overloads sufficient to ensure, for a call with at least one argument of type `complex<T>`:

- ~~If either argument has type `complex<long double>` or type `long double`, then both arguments are effectively cast to `complex<long double>`.~~
- ~~Otherwise, if either argument has type `complex<double>`, `double`, or an integer type, then both arguments are effectively cast to `complex<double>`.~~
- ~~Otherwise, if either argument has type `complex<float>` or `float`, then both arguments are effectively cast to `complex<float>`.~~
- If one argument is of type `T1` or `complex<T1>` and the other argument is of type `T2` or `complex<T2>` where `T1` and `T2` are both floating-point types:
  - If the floating-point conversion ranks (`[conv.rank]`) of `T1` and `T2` are different and unordered, the program is ill-formed.
  - Otherwise, if `T1` has greater floating-point conversion rank than `T2`, then both arguments are effectively cast to `complex<T1>`.
  - Otherwise, both arguments are effectively cast to `complex<T2>`.
- Otherwise, if the other argument has integer type, it is effectively cast to `complex<T>`.

Note: No literal suffixes are defined for complex numbers of extended floating-point types. Subclause `[complex.literals]` is unchanged.

## § 6.6. `<atomic>`

Change the wording so that the specializations of `std::atomic` for floating-point types apply to all floating-point types, not just the standard floating-point types listed.



The specializations of `std::atomic` for integral types are not required to include specializations for all extended integral types, only for the extended types that are used in `<cstdint>`. It would be reasonable for this proposal to adopt a similar approach. If we take that approach, there are no wording changes to `<atomic>` in this paper. Instead, there would be some changes to `<atomic>` as part of [P1468], requiring specializations only for the floating-point aliases that name extended floating-point types.

**ISSUE 5** Should `std::atomic` have specializations for all floating-point types, or only for extended floating-point types with well-known aliases (see [P1468])?

### § 6.6.1. Wording

Modify 31.8.3 "Specializations for floating-point types" [`atomics.types.float`] paragraph 1 as follows:

There are specializations of the `atomic` class template for ~~the~~ **all** floating-point types ~~`float`,~~  
~~`double`, and `long double`~~. For each such type *floating-point*, the specialization `atomic<floating-point>` provides additional atomic operations appropriate to floating-point types.

### § 6.7. Feature test macro

No feature test macro is being proposed for the library changes in this paper. The library changes would be covered by the core language [feature test macro](#), if there is one.

## § References

### § Informative References

#### [P0192]

Michał Dominiak; et al. [`short float` and fixed-size floating point types](https://wg21.link/P0192). URL: <https://wg21.link/P0192>

#### [P1468]

Michał Dominiak; Boris Fomitchev; Sergei Nikolaev. [Fixed-layout floating-point type aliases](https://wg21.link/P1468). URL: <https://wg21.link/P1468>

## § Issues Index

**ISSUE 1** Should implicit conversions be allowed from larger floating-point types to smaller floating-point types? [↵](#)

**ISSUE 2** Should there be a feature test macro to indicate that the implementation supports at least one extended floating-point type? [↵](#)

**ISSUE 3** Should the new type traits `std::is_standard_floating_point` and/or `std::is_extended_floating_point` be introduced? [↵](#)

**ISSUE 4** Should a new type trait be introduced that can be used to query the floating-point conversion rank relationship? [↵](#)

**ISSUE 5** Should `std::atomic` have specializations for all floating-point types, or only for extended floating-point types with well-known aliases (see [\[P1468\]](#))? [↵](#)